

UNSA

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA

PROYECTO INTEGRADOR

Aplicación cloud-native en AWS: CI/CD, IaC, Kubernetes y monitoreo

Curso: Cloud Computing

Johan Lizarve Mamani
Sebastian Mendoza Fernandez
Sergio Ahmed Castillo Sancho

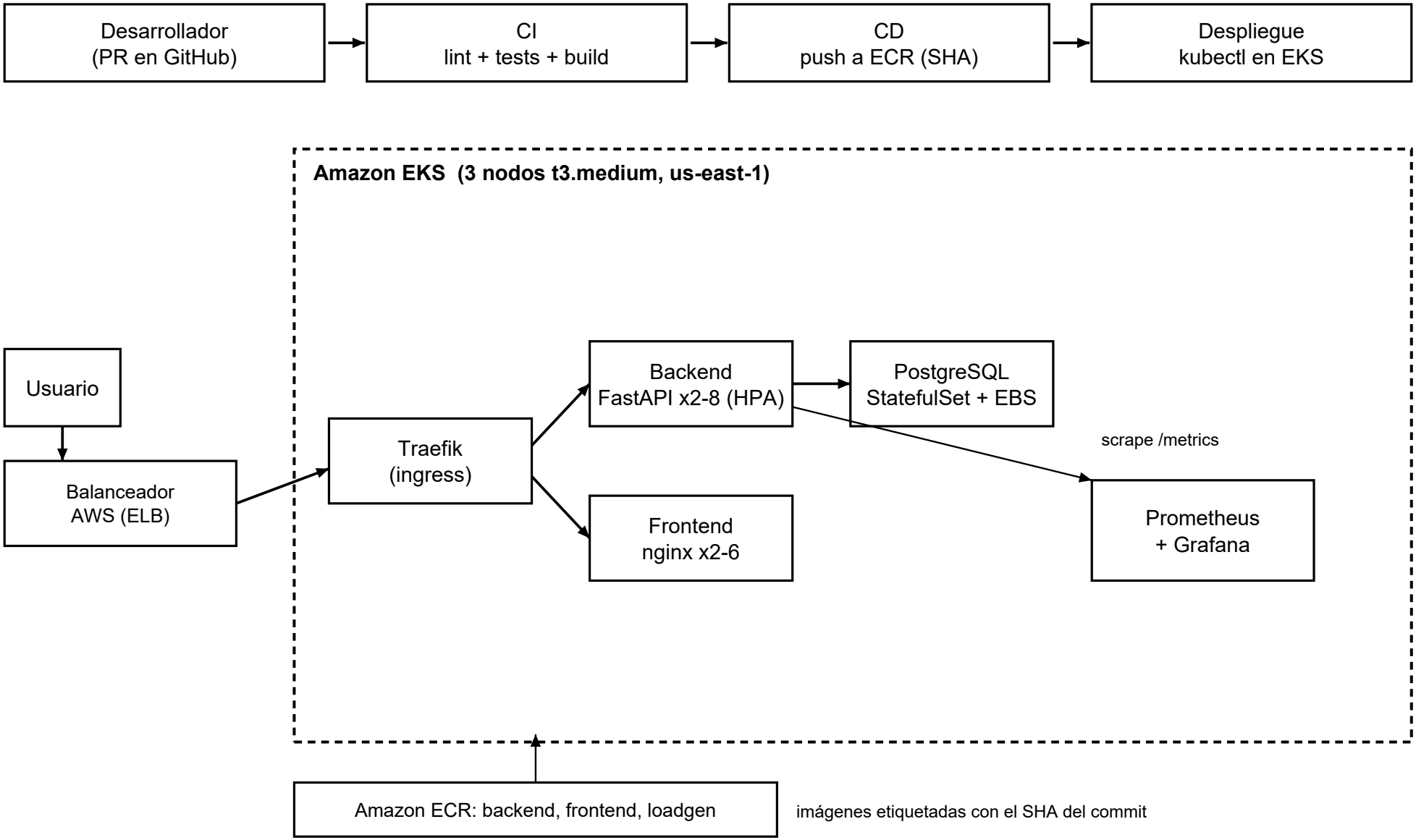
Arequipa, noviembre de 2026

Qué problema resolvimos

- Partimos de una app de laboratorio: exchange de apuestas (FastAPI + nginx + PostgreSQL) que solo corría en *kind* local.
- Objetivo: llevarla a un ciclo **cloud-native completo en AWS**.
- Propiedad que protegemos siempre: **drift = 0** (saldo de la billetera = suma del libro de movimientos).
- Liquidación idempotente: un reintento no paga dos veces.

Arquitectura y flujo

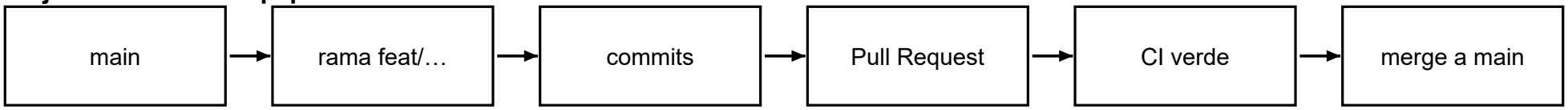
Flujo de entrega



Git y flujo de trabajo (4.1)

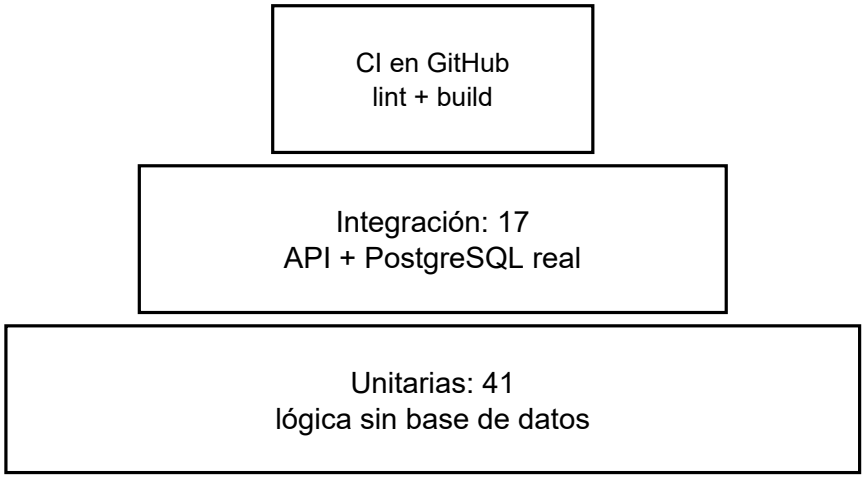
- Fork propio, rama *feat/tests-and-ci*, Pull Request #1 a main (5 commits, 38 archivos).
- Una rama por tarea; solo se fusiona con el CI en verde.
- Documentación: README, runbook, tutorial del equipo, informe.

Flujo de ramas del equipo

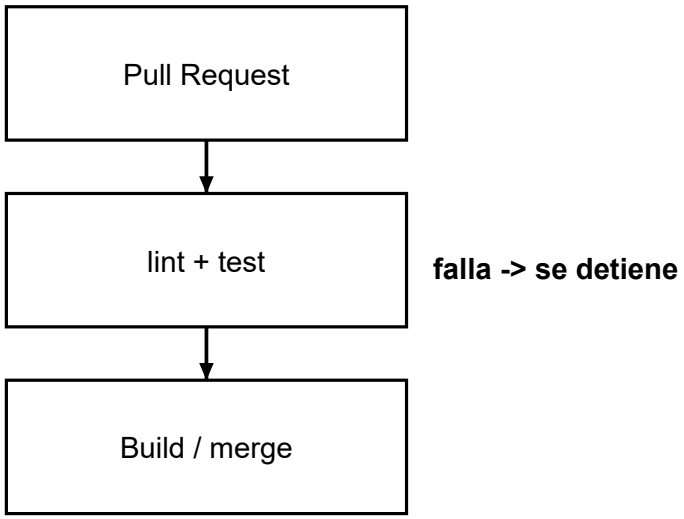


Testing (4.2)

- Unitarias (41): redondeo, rate limit, circuit breaker, sondas, validación, métricas.
- Integración (17): API con PostgreSQL real, liquidación idempotente, drift = 0.
- **REQUIRE_DB=1**: si falta la base de datos, la prueba falla, no se omite.



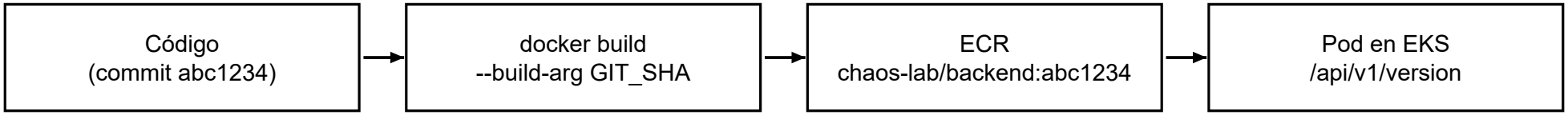
Pirámide de pruebas del proyecto



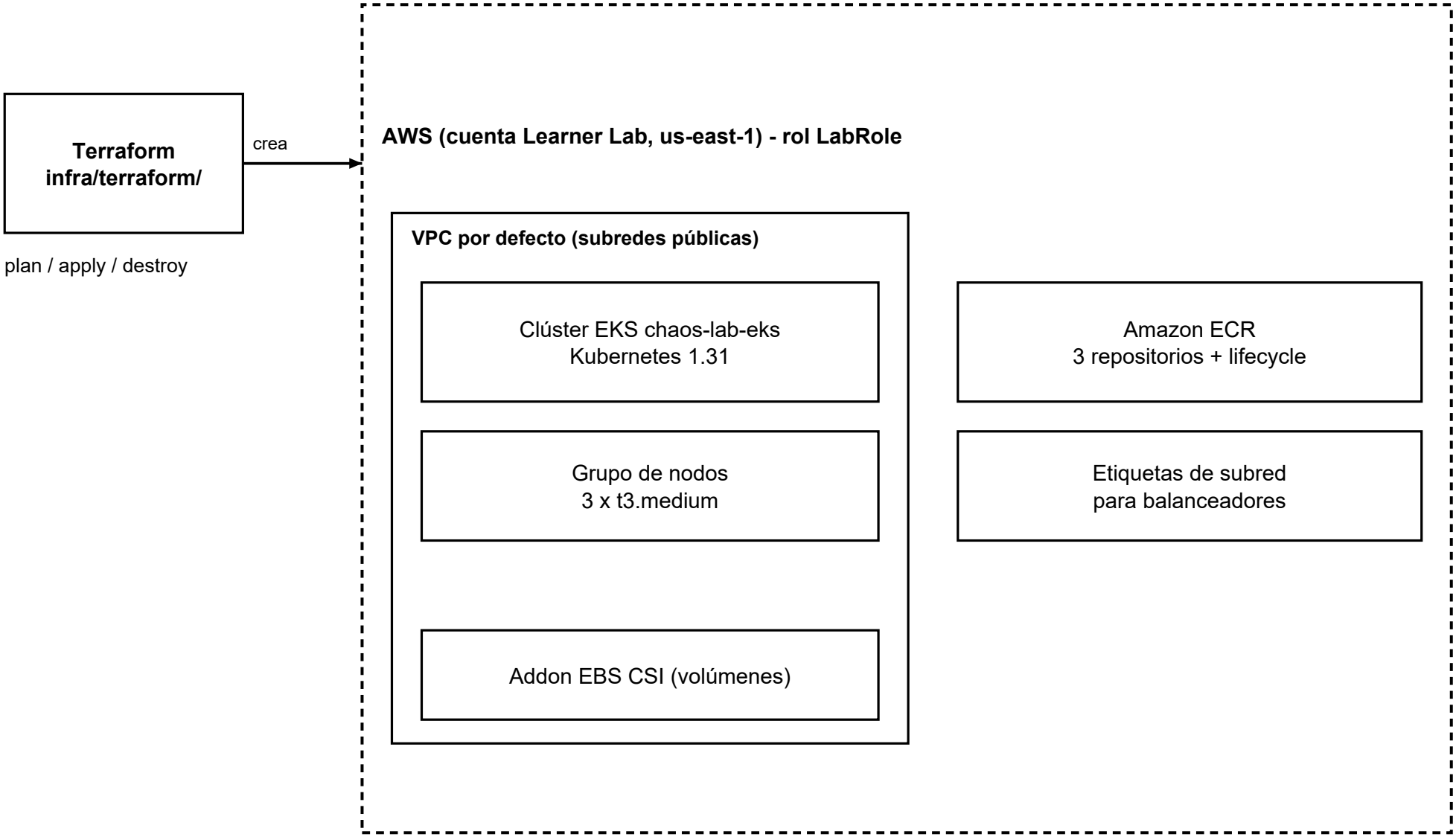
Contenedores y registry (4.3)

- Dockerfiles multi-stage y sin usuario root (backend, frontend, loadgen).
- Amazon ECR con limpieza de imágenes antiguas.
- Versionado por **SHA del commit**, nunca *latest*.
- **GET /api/v1/version** devuelve el SHA: se ve en vivo qué versión corre.

Trazabilidad commit -> imagen -> pod

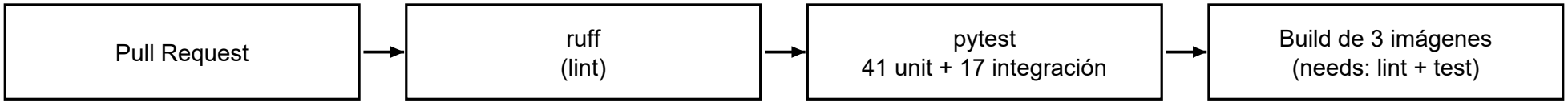


Infraestructura como código (4.4)



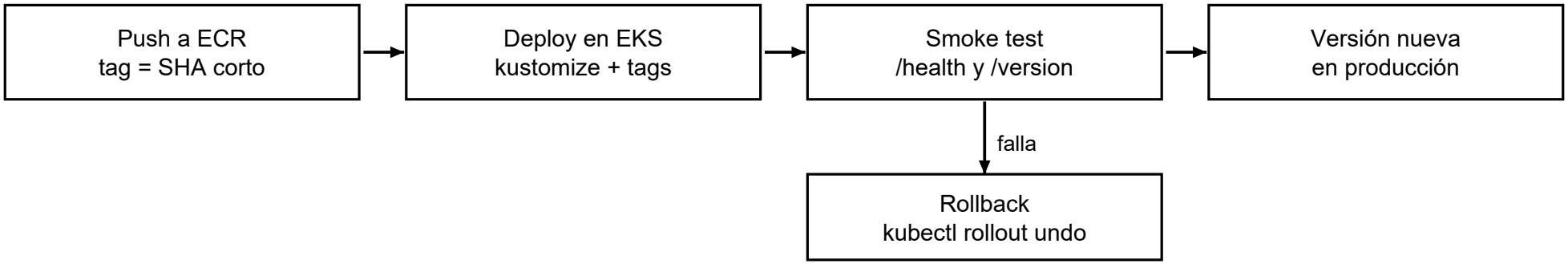
CI y CD (4.5 y 4.6)

Integración continua (en cada PR)



Si un test falla: el build no se ejecuta (pipeline en rojo)

Entrega continua (CI en verde sobre main)



Kubernetes (4.7)

Namespace de la aplicación

Deployment backend
2-8 réplicas (HPA)
requests y limits
PodDisruptionBudget

Deployment frontend
2-6 réplicas
nginx

StatefulSet PostgreSQL
volumen EBS (PVC)
secretos aleatorios

Sondas del backend

startup
da tiempo de arranque

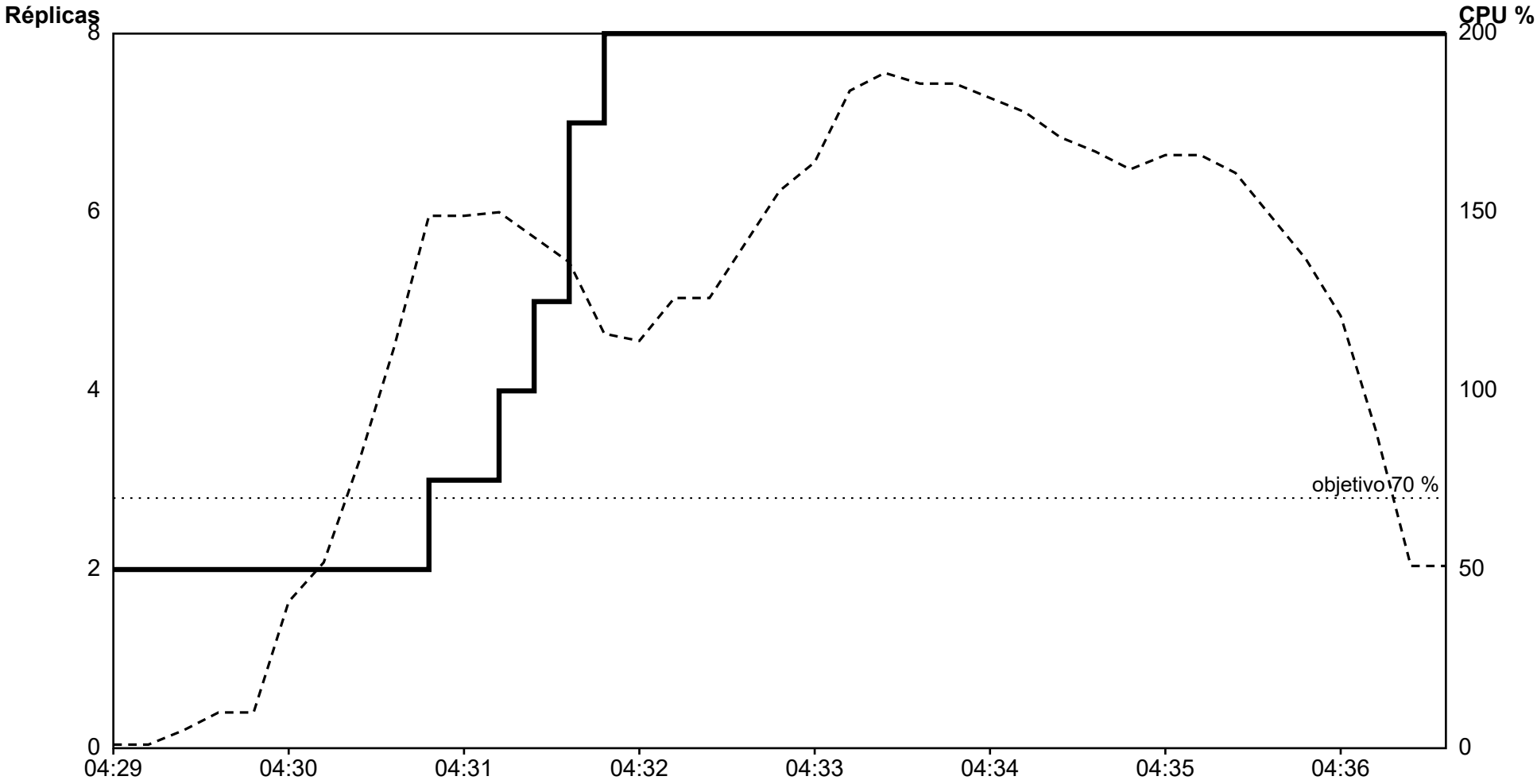
liveness
no consulta la base de datos

readiness
sí consulta la BD: si cae, sale del Service

Recuperación: se elimina un pod -> el Deployment lo recrea; drift sigue en 0

Resultado real: HPA bajo carga (4.8)

- **2 a 8 réplicas en 64 s** (04:30:53 a 04:31:57); CPU pico 189 %; memoria estable (19-21 %).



Línea continua: réplicas disponibles. Discontinua: CPU media (%). Punteada: objetivo 70 %. Datos medidos (docs/evidencias/hpa.csv).

Resultado real: prueba de carga k6 (4.8)

Métrica	**Valor medido**
Perfil	ramp, 8 min, hasta 300 usuarios virtuales
Peticiones	218 929 (454 por segundo)
Latencia media / p90 / p95	190 ms / 276 ms / 526 ms
Latencia máxima	3,2 s
Checks superados	100 % (218 913)
Peticiones fallidas	0,86 % (incluye 402 esperados)
Umbrales p95 < 1500 ms y error < 5 %	Cumplidos

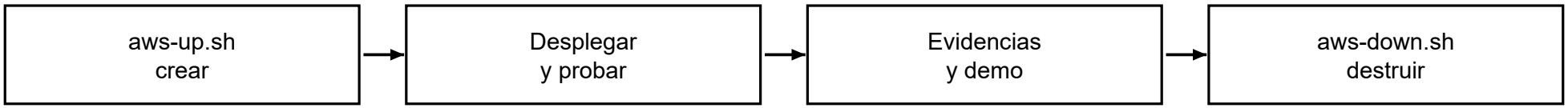
Resultado real: recuperación ante fallos (4.7)

Acción	Resultado medido
Eliminar un pod del backend con tráfico	47 peticiones OK, 0 fallidas
Eliminar PostgreSQL	Un 503 transitorio; recuperado en ~25 s
Reinicios del backend por esa causa	0
Consistencia del dinero	drift = 0 (también tras la carga)

Seguridad y costos

- Sin credenciales en el repositorio; secretos generados al desplegar.
- Limitación: Learner Lab no permite crear roles IAM; se usa LabRole (sin mínimo privilegio completo).
- Costo: **≈ 0,25 USD por hora**; se crea solo para pruebas y demo.
- Destrucción verificada con **aws-down.sh**.

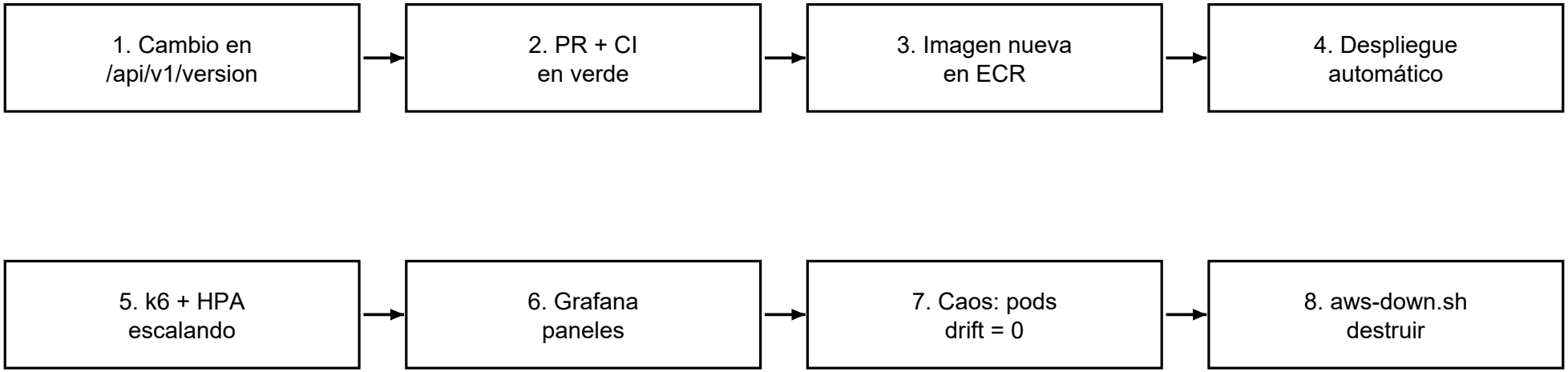
Ciclo de vida del entorno (control de costo)



Estado y evidencias

Elemento	Resultado medido
Tests, CI, build, terraform plan	41 + 17 pruebas, CI verde
EKS y ECR en AWS	Clúster ACTIVE, 3 nodos, imágenes por SHA
CD (GitHub Actions)	Publish, deploy y smoke test en verde
Caos	0 peticiones fallidas al matar un pod; drift = 0
k6 (300 VU, 8 min)	454 req/s, p95 526 ms
HPA	2 a 8 réplicas en ~1 min

Demo en vivo


1. Cambio en /api/v1/version
2. PR + CI en verde
3. Imagen nueva en ECR
4. Despliegue automático
5. k6 + HPA escalando
6. Grafana paneles
7. Caos: pods drift = 0
8. aws-down.sh destruir

Conclusiones y mejoras

- Se cubre el ciclo completo: código, pruebas, CI/CD, IaC, Kubernetes, autoescalado y monitoreo.
- Mejoras: backend remoto de Terraform, OIDC en lugar de claves, NetworkPolicy activas, alertas, subredes privadas.
- Hallazgos de la app: los 500 no controlados no se cuentan en métricas; hay un return inalcanzable en settle_bet.

Anexo: repositorios

Repositorio	Dirección
Proyecto (fork del equipo)	https://github.com/sergioandante/bet-exchange-chaos-lab-sergio
Pull Request principal	https://github.com/sergioandante/bet-exchange-chaos-lab-sergio/pull/1
Repositorio base	https://github.com/helvetios16/bet-exchange-chaos-lab